Case Study: Use of the Rabin–Karp Algorithm in Plagiarism Detection

Name: Rohit Sharma
Roll No: 7445
Class: BE Computer B

Presented to: Prof. Rushali Katkar

Why Plagiarism Detection Matters

Academic integrity

Essays and reports are copied, lightly edited and resubmitted. Institutions need a fair and consistent way to flag them.

Code originality

Programs are copied with renamed variables or reordered statements, which is hard to spot by reading.

Scale

Every new submission must be checked against a growing archive of earlier work and web sources.

Manual checking is impractical, so detection must be automatic, fast and tolerant of small edits.

Why Naive Matching Does Not Scale

	Naive matching	Hash-based (Rabin–Karp)
What is compared	Characters of every window	One number per window
Cost per window	Up to O(m) character checks	O(1) hash update
Over a text of n characters	O(nm) in the worst case	O(n) expected for hashing
Stored per document	The full text	A compact set of numbers

5×10^{11}

document pairs to compare directly when there are 1,000,000 documents

Comparing full texts pairwise cannot work at this scale. Fingerprints reduce each document to a small set of numbers that can be looked up in an index.

Recap: The Rabin–Karp Rolling Hash

Rolling hash update

$$H(\text{next}) = (d \times (H - t[i] \times d^{m-1}) + t[i+m]) \bmod q$$

H: hash of the current window d: base

m: window length q: a large prime

t[i]: character leaving the window

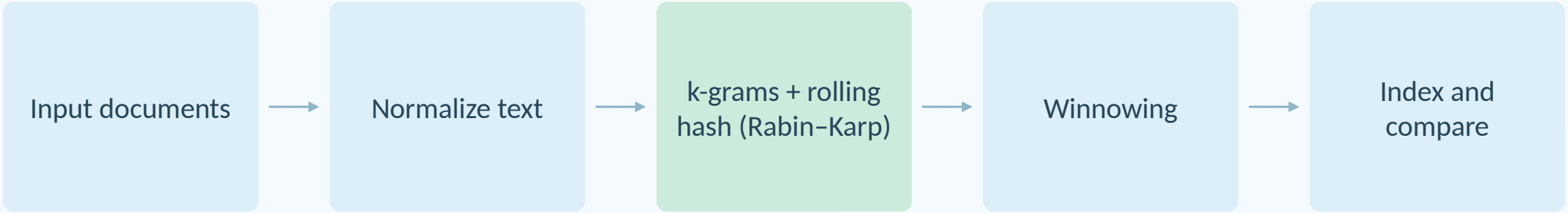
t[i+m]: character entering the window

Cost

- Update: O(1) per slide
- Expected time: O(n + m)
- Worst case: O(nm), when collisions are frequent
- Equal hashes are verified character by character

Plagiarism tools reuse exactly this idea: every substring of a fixed length can be hashed in constant time.

From Pattern Search to Document Fingerprinting



Classic Rabin–Karp

One given pattern is searched for inside one text.

Plagiarism detection

No pattern is given. Every document is reduced to a fingerprint set, and the sets are compared with each other.

Step 1: Normalize the Text

Original text

```
A do run run run, a do run run
```


Normalized text

```
adorunrunrunadorunrun
```

Normalization rules

- Convert everything to lowercase
- Remove punctuation and whitespace
- For source code: remove comments and replace variable names with a common token
- Goal: small cosmetic edits must not change the k-grams

Step 2: k-grams and the Rolling Hash (k = 5)

#	k-gram	Hash (d = 256, q = 101)
1	adoru	8
2	dorun	80
3	orunr	26
4	runru	50
5	unrun	19
6	nrunr	59
7	runru	50

One rolling step

$H(\text{"adoru"}) = 8$

Remove 'a' (97), shift, add 'n' (110):

$$(256 \times (8 - 97 \times 256^4) + 110) \bmod 101$$

$$= 80 = H(\text{"dorun"})$$

Same value as hashing the new k-gram from scratch, at O(1) cost.

Step 3: Winnowing Selects the Fingerprints

1. Hash every k-gram using the rolling hash.
2. Slide a window of w consecutive hashes over the list.
3. Keep the smallest hash in each window (rightmost if tied).
4. The kept (hash, position) pairs form the document's fingerprint.

Guarantee

$w + k - 1$

Any copied passage at least this long produces a shared fingerprint.

Density

$\approx 2 / (w + 1)$

Expected fraction of hashes that are kept, so storage stays small.

Worked Example: Winnowing (k = 5, w = 4)

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
8	80	26	50	**19**	59	50	**19**	42	**24**	30	47	**8**	80	26	50	**19**

`Text: "adorunrunrunadorunrun"  (17 hashes, 14 windows of 4)`

Highlighted hashes are the minimum of at least one window.

Fingerprint (hash @ position)

`8 @ 1` `19 @ 5` `19 @ 8` `24 @ 10` `8 @ 13` `19 @ 17`

6 of 17 hashes are stored (35%), yet every repeated passage still leaves a trace.

Step 4: Index and Compare Documents

Fingerprint hash	Found in
8	Doc A, Doc B
19	Doc A, Doc B
24	Doc A, Doc B

```
Doc A: "adorunrunrunadorunrun"
Doc B: "dorunrunrunadorun"
```

How a new submission is checked

- Look up each fingerprint in an inverted index (hash → documents).
- Doc B has 3 distinct fingerprints; 3 also appear in Doc A.
- Similarity: 3 / 3 = 100% of B's fingerprints.
- Matching positions are mapped back to the text to highlight copied passages.

Real-World Systems

System	Used for	Technique
MOSS (Stanford)	Source code in many programming languages	Winnowing over k-gram hashes
JPlag	Source code submissions in courses	Greedy String Tiling accelerated with Karp–Rabin hashing (RKR-GST)
Turnitin	Student essays and reports	Proprietary; the exact method is not public

A reported match is evidence for a human reviewer, not a verdict.

Complexity Analysis

Stage	Time	Remark
Normalization	O(n)	Single pass over the text
k-gram hashes (rolling)	O(n)	O(1) per update, versus O(nk) if each k-gram is hashed afresh
Winnowing	O(n)	Sliding-window minimum; O(nw) if each window is scanned
Index lookup	O(f) expected	$f \approx 2n / (w + 1)$ fingerprints per document
Verifying a match	O(length)	Only for flagged regions, to rule out collisions

Overall: expected O(n) per document, with storage reduced to about $2n / (w + 1)$ numbers.

Limitations and Challenges

Paraphrasing and obfuscation

Rewording, translation, or restructured code defeat exact k-gram matching.

Common text

Quotations, references and standard phrases create matches that are not plagiarism.

Hash collisions

Different strings can share a hash, so matches must be verified.

Choosing k and w

Small k gives noise; large k or w misses short copied passages.

Conclusion

1. **The rolling hash makes fingerprinting linear.**
   Every k-gram hash costs O(1), so a document is processed in O(n).

2. **Winnowing keeps the data small.**
   A guaranteed subset of hashes is stored, so millions of documents can be indexed.

3. **Humans make the final decision.**
   Collisions and common text mean matches must be reviewed.